

**INSTITUTO FEDERAL CATARINENSE - IFC
BACHARELADO DE CIÊNCIA DA COMPUTAÇÃO**

ANDRÉ VITOR BASTOS DE MACÊDO

PATHFINDING EM MALHAS 3D : Análise estatística e comparativa

**Blumenau
2026**

ANDRÉ VITOR BASTOS DE MACÊDO

PATHFINDING EM MALHAS 3D : Análise estatística e comparativa

Pré-projeto de Trabalho de Conclusão de Curso
apresentado ao curso de Bacharelado em Ciência
da Computação do Instituto Federal Catarinense,
como requisito parcial para a obtenção do grau de
Bacharel.

Orientador(a): Prof. Paulo César Rodacki Gomes

LISTA DE ILUSTRAÇÕES

Figura 1	Diagrama de classes do padrão <i>Composite</i>	12
Figura 2	Diagrama de classes do padrão <i>Singleton</i>	13
Figura 3	Etapas de geração de um labirinto perfeito utilizando o algoritmo de Kruskal	16
Figura 4	Exemplo de mapa de altura	17
Figura 5	Conversão de um mapa de altura para uma malha 3D	18
Figura 6	Exemplo de malha gerada a partir de um mapa de altura	19
Figura 7	Exemplo de ruído de Perlin e um de seus <i>octaves</i>	20
Figura 8	Ruídos de Perlin com diferentes tamanhos de células	21
Figura 9	Cálculo do produto escalar em uma célula do ruído de Perlin	22
Figura 10	Ruídos de Perlin com diferentes frequências e amplitudes	22
Figura 11	Exemplo de malha gerada a partir de um ruído de Perlin	24
Figura 12	Função de Suavização no Ruído de Perlin	25
Figura 13	Diagrama de Thread Pool e filas multinível	27
Figura 14	Diagrama de classe do Composite	29

LISTA DE TABELAS

Tabela 1 Cronograma de execução das atividades	32
--	----

LISTA DE ABREVIATURAS E SIGLAS

A*	<i>A-Star</i>
API	<i>Application Programming Interface</i> (Interface de Programação de Aplicações)
CPU	<i>Central Processing Unit</i> (Unidade Central de Processamento)
EBO	<i>Element Buffer Object</i>
GLFW	<i>Graphics Library Framework</i>
GPU	<i>Graphics Processing Unit</i> (Unidade de Processamento Gráfico)
IFCG	Instituto Federal Catarinense/Computação Gráfica
JPS	<i>Jump Point Search</i>
MST	<i>Minimum Spanning Tree</i> (Árvore Geradora Mínima)
OpenGL	<i>Open Graphics Library</i>
RAII	<i>Resource Acquisition Is Initialization</i>
RGB	<i>Red, Green, Blue</i>
SIGSEGV	<i>Segmentation Violation</i> (Violação de Segmentação)
STL	<i>Standard Template Library</i>
VAO	<i>Vertex Array Object</i>
VBO	<i>Vertex Buffer Object</i>

SUMÁRIO

1 INTRODUÇÃO	7
2 JUSTIFICATIVA	8
3 OBJETIVOS	9
3.1 Objetivos Gerais	9
3.2 Objetivos Específicos	9
4 FUNDAMENTAÇÃO TEÓRICA	10
4.1 Pilha Tecnológica	10
4.1.1 Linguagem de Programação	10
4.1.2 Arquitetura de Programas	11
4.2 Teoria dos Grafos	13
4.2.1 Definição e Propriedades dos Grafos	13
4.2.2 Grafos Direcionados e Não Direcionados	13
4.2.3 MST (<i>Minimum Spanning Tree</i>)	13
4.3 Computação Gráfica	13
4.3.1 OpenGL API	14
4.4 Geração de Cenários de Teste	15
4.4.1 Labirintos	15
4.4.2 Mapas de Altura	16
4.4.3 Geração de Ruídos	19
4.5 Algoritmos de Busca Heurística	26
4.5.1 Dijkstra	26
4.5.2 Heurísticas	26
4.5.3 A*	26
4.5.4 Jump Point Search (JPS)	26
4.5.5 Theta*	26
4.6 Concorrência e Paralelismo	26
4.6.1 <i>Multithreading</i>	26
4.6.2 Monitores	26
4.6.3 Fila Multinível e <i>Thread Pool</i>	26
5 METODOLOGIA	28
5.1 Arquitetura do Motor Gráfico (IFCG)	28
5.1.1 Aplicação de conceitos	28
5.1.2 Estrutura de Renderização e Gerenciamento de Recursos	28
5.1.3 Motor de Renderização e seus Componentes	30
5.2 Implementação da Estrutura de Grafos	30
5.2.1 Representação de Grafos Direcionados e Não Direcionados	30
5.2.2 Implementação de Algoritmos de Busca em Grafos	30
5.2.3 Otimizações e Estruturas de Dados Auxiliares	30
5.3 Geração e Processamento dos Cenários	30

5.3.1	Geração de Malhas 3D Complexas	30
5.3.2	Processamento de Malhas e Extração de Grafos	30
5.3.3	Configuração de Cenários de Teste e Parâmetros	30
5.4	Implementação dos Algoritmos e Sistema Concorrente	30
5.4.1	Implementação de Algoritmos de Pathfinding	30
5.4.2	Adaptação para Ambientes 3D	30
5.4.3	Implementação de Multithreading e Testes de Estresse	30
5.5	Desenho Experimental e Coleta de Dados	31
5.5.1	Configuração dos Testes e Métricas Coletadas	31
5.5.2	Execução dos Testes e Registro dos Resultados	31
5.5.3	Análise Estatística e Visual dos Resultados	31
6	CRONOGRAMA	32

1 INTRODUÇÃO

2 JUSTIFICATIVA

Com este capítulo, pretende-se justificar a relevância e a necessidade do estudo proposto, destacando a importância de uma análise estatística detalhada dos algoritmos de pathfinding em malhas 3D, especialmente considerando as complexidades e desafios dos ambientes reais.

Na área de algoritmos de *pathfinding*, busca de caminhos, a maioria dos estudos se concentra em topologias 2D ou não geométricas e em cenários ideais, onde as condições são controladas e otimizadas para destacar as vantagens de cada algoritmo. No entanto, a transição para ambientes 3D introduz uma série de desafios adicionais, como a complexidade da geometria, a necessidade de lidar com obstáculos tridimensionais e a gestão de recursos computacionais. A falta de análises estatísticas detalhadas sob condições adversas limita a compreensão real do desempenho desses algoritmos em situações práticas, onde otimizações matemáticas e técnicas avançadas podem ter um impacto significativo.

Este estudo se diferencia por adotar uma abordagem aprofundada e detalhada, focando em métricas quantitativas e condições adversas que refletem os desafios do mundo real. Ao invés de uma visão superficial e generalizada, a pesquisa se propõe a investigar o comportamento de algoritmos de busca quando expostos a concorrência computacional e arquiteturas complexas.

Além disso, a construção de uma infraestrutura gráfica e de uma biblioteca de grafos do zero não apenas proporciona um ambiente de teste personalizado, mas também garante um controle total sobre as variáveis e anomalias de hardware que podem afetar os resultados. A correta aplicação teórica e estrutural da base de grafos é fundamental para garantir a validade dos testes empíricos, conforme destacado por Gomes (2022).

Com isso, viu-se a necessidade de um estudo que vá além dos testes em ambientes ideais, oferecendo uma análise estatística detalhada do desempenho dos algoritmos de pathfinding em malhas 3D, contribuindo para a escolha informada de técnicas de navegação em projetos futuros.

3 OBJETIVOS

Neste capítulo serão apresentados os objetivos gerais e específicos do estudo, detalhando o que se pretende alcançar com a pesquisa. As metas girarão em torno da implementação de algoritmos de pathfinding, desenvolvimento de uma infraestrutura gráfica para testes, coleta e análise de dados, e apresentação dos resultados.

3.1 Objetivos Gerais

Analisar comparativamente o desempenho dos algoritmos de pathfinding como A*, JPS e Theta* em ambientes tridimensionais, considerando métricas de execução, concorrência e técnicas de otimização.

3.2 Objetivos Específicos

1. Implementar uma estrutura de dados eficiente para representação de grafos direcionados e não direcionados voltada à execução de algoritmos de pathfinding.
2. Desenvolver uma infraestrutura de renderização para visualização de malhas tridimensionais e execução integrada dos algoritmos de pathfinding.
3. Implementar e adaptar algoritmos heurísticos (A*, JPS, Theta* e outros) para ambientes tridimensionais.
4. Adaptar a arquitetura do sistema para execução concorrente e realização de testes de estresse utilizando multithreading.
5. Desenvolver geradores de malhas topológicas complexas para simulação de diferentes cenários de teste.
6. Coletar e analisar métricas de desempenho dos algoritmos de pathfinding, incluindo tempo de execução, uso de memória e qualidade dos caminhos gerados.
7. Elaborar análises estatísticas e visuais comparativas dos resultados obtidos nos testes realizados.

4 FUNDAMENTAÇÃO TEÓRICA

Para a realização deste estudo, foi necessário adquirir conhecimentos em diversas áreas, incluindo algoritmos de busca, estruturas de dados, computação gráfica e concorrência. A seguir, são detalhados os principais tópicos que compõem a base teórica deste trabalho.

4.1 Pilha Tecnológica

4.1.1 Linguagem de Programação

Todas as implementações e experimentos deste estudo foram realizados utilizando a linguagem de programação C++20. Escolhida por sua eficiência, controle de baixo nível e ampla adoção na indústria de jogos e simulações, a linguagem C++ oferece recursos avançados para manipulação de memória, concorrência e otimização de desempenho, essenciais para o desenvolvimento de algoritmos de pathfinding em ambientes tridimensionais.

A escolha do C++20 também se justifica pela disponibilidade de bibliotecas e *frameworks* que facilitam a implementação de estruturas de dados complexas, renderização gráfica e multithreading, além de permitir uma integração eficiente com APIs gráficas como OpenGL.

Enquanto a implementação da infraestrutura gráfica e dos algoritmos de pathfinding foi realizada do zero, a escolha do C++20 garantiu um ambiente de desenvolvimento robusto e flexível, permitindo a aplicação de técnicas avançadas de otimização e controle total sobre os recursos computacionais utilizados durante os testes.

4.1.1.1 Estruturas de Dados

A implementação de algoritmos de pathfinding em malhas 3D requer o uso de estruturas de dados eficientes para representar grafos, filas de prioridade e outras estruturas auxiliares. Para isso, foi utilizada a biblioteca padrão do C++20, a STL (Standard Template Library), que oferece uma variedade de contêineres e algoritmos otimizados para manipulação de dados.

A escolha de estruturas de dados adequadas é crucial para garantir o desempenho dos algoritmos, especialmente em ambientes tridimensionais onde a complexidade pode aumentar significativamente. Por isso foram utilizadas estruturas como vetores, mapas de *hash* e filas de prioridade para representar os grafos e gerenciar os nós durante a execução dos algoritmos de busca.

4.1.1.2 Funções *Lambda*

A introdução de funções *lambda* em C++11 e suas melhorias contínuas nas versões subsequentes, incluindo C++20, proporcionaram uma maneira mais concisa

de definir funções anônimas. No contexto deste estudo, as funções *lambda* foram amplamente utilizadas.

Por definição, uma função anônima é uma função “sem nome”, que pode ser definida e utilizada diretamente no local onde é necessária. Essas funções são particularmente úteis para operações que exigem uma função de curto prazo, como a definição de heurísticas em algoritmos de busca ou a implementação de *callbacks* para eventos específicos durante a renderização ou execução dos algoritmos.

4.1.1.3 Biblioteca *jthread*

Um dos principais motivadores para o uso da versão C++20 é a introdução da biblioteca *jthread*, que simplifica significativamente a implementação de multithreading e concorrência. A *jthread* oferece uma interface mais segura e fácil de usar para gerenciamento de *threads*, incluindo a capacidade de interromper *threads* de forma cooperativa, o que é crucial para a realização de testes de estresse e simulações em ambientes dinâmicos.

Esta biblioteca utiliza dos princípios de RAII (Resource Acquisition Is Initialization) para garantir que os recursos sejam gerenciados de forma eficiente e segura, evitando problemas comuns em multithreading, como o encerramento abrupto do programa devido a *threads* não finalizadas ao saírem de escopo. A classe de *thread* é gerenciada de uma forma que, perante sua destruição, a *thread* faz um *join* automaticamente, garantindo que os recursos sejam liberados corretamente. Isso não só facilita a implementação de concorrência, mas também melhora a segurança do código.

Um *join* é uma operação que bloqueia a execução da *thread* chamadora até que a *thread* alvo termine sua execução. No caso da *jthread*, o *join* é automático, o que significa que quando um objeto de *jthread* é destruído, ele automaticamente chama o método *join()*, garantindo que a *thread* associada seja finalizada corretamente antes de liberar os recursos.

Também, a introdução dos *stop tokens* na biblioteca permite que as *threads* sejam interrompidas de maneira cooperativa, o que é especialmente útil para simulações e testes de estresse, onde é necessário controlar o tempo de execução das *threads* e garantir que elas possam ser finalizadas de forma segura.

4.1.2 Arquitetura de Programas

A arquitetura das bibliotecas e programas desenvolvido para este estudo foi projetada para ser modular e escalável, permitindo a fácil integração de novos algoritmos de pathfinding e a adaptação a diferentes cenários de teste. Por isso foi escolhida uma arquitetura orientada a objetos, que facilita a organização do código e a reutilização de componentes.

Essa escolha se dá pela necessidade de criar uma infraestrutura gráfica personalizada e uma estrutura de dados eficiente para representar grafos, além de implementar algoritmos de pathfinding que possam ser facilmente adaptados e otimizados

para ambientes tridimensionais. A arquitetura orientada a objetos permite encapsular a complexidade dos diferentes componentes do sistema, como a renderização gráfica, a representação de grafos e a execução dos algoritmos, facilitando a manutenção e evolução do código ao longo do desenvolvimento do estudo.

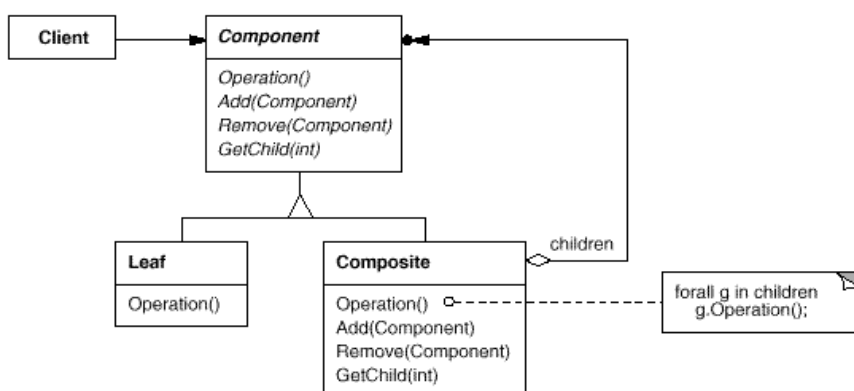
Em adição, o uso de uma arquitetura orientada a objetos permite abstrações de hierarquia e polimorfismo, que serão amplamente utilizadas na implementação de ambas as principais bibliotecas, tanto a de grafos quanto a de renderização gráfica.

4.1.2.1 Padrões de Projeto

Padrões de projeto são soluções reutilizáveis para problemas comuns de design de *software* (Gamma et al., 1994). No desenvolvimento deste estudo, foram aplicados diversos padrões de projeto para garantir a modularidade, flexibilidade e manutenibilidade do código.

O padrão *Composite* (Gamma et al., 1994) foi utilizado para representar a hierarquia de objetos na infraestrutura gráfica, permitindo que objetos complexos sejam tratados de forma uniforme, e na representação de grafos, onde grafos direcionados e não direcionados podem ser manipulados de maneira consistente. Neste padrão, objetos compostos (*Composite*) e objetos individuais (*Leaves* ou folhas) são tratados de forma uniforme, com a introdução de um parente comum (*Component*) que define a interface para ambos. Com isso, é possível que *Composite* delegue a execução de uma função comum entre eles para todos os seus filhos, sejam eles *Composite* ou *Leaves*.

Figura 1 – Diagrama de classes do padrão *Composite*

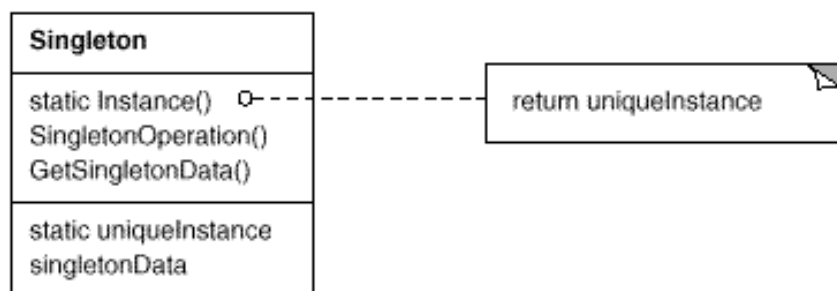


Fonte: Gamma et al. (1994)

Em adição, foi utilizado o padrão *Singleton* (Gamma et al., 1994). Este padrão dita que uma classe deve ter apenas uma instância e fornecer um ponto global de acesso a essa instância. Para isso, o construtor da classe é privado e ela contém uma variável estática que armazena a única instância da classe (inicializada como nula). A classe também tem um método público estático que retorna a instância, criando-a

se ainda não existir. Este padrão é útil para gerenciar recursos compartilhados, como o contexto gráfico do OpenGL, garantindo que haja um controle centralizado sobre a renderização e os recursos gráficos utilizados durante os testes.

Figura 2 – Diagrama de classes do padrão *Singleton*



Fonte: Gamma et al. (1994)

4.2 Teoria dos Grafos

Uma vez que os algoritmos de pathfinding operam sobre estruturas de grafos para encontrar caminhos entre nós, o estudo da teoria dos grafos é a base fundamental deste experimento. A teoria dos grafos fornece as ferramentas e conceitos necessários para representar e manipular essas estruturas, permitindo a implementação eficiente dos algoritmos de busca.

Segundo Gomes (2022), um grafo é uma estrutura de dados composta por um conjunto de vértices (ou nós) e um conjunto de arestas (ou ligações) que conectam esses vértices. Os grafos podem ser direcionados, onde as arestas têm uma direção específica, ou não direcionados, onde as arestas não possuem direção. A representação de grafos pode ser feita de diversas formas, como listas de adjacência, matrizes de adjacência ou listas de arestas, cada uma com suas vantagens e desvantagens em termos de eficiência e uso de memória.

4.2.1 Definição e Propriedades dos Grafos

4.2.2 Grafos Direcionados e Não Direcionados

4.2.3 MST (*Minimum Spanning Tree*)

4.3 Computação Gráfica

Por causa da necessidade de controle total sobre a infraestrutura gráfica e a implementação dos algoritmos, optou-se por desenvolver um monitor gráfico do zero,

denominada IFCG¹ (Instituto Federal Catarinense/Computação Gráfica). Esta decisão foi motivada pela necessidade de um ambiente de teste personalizado, que permita a coleta de dados em condições controladas e a aplicação de otimizações específicas para os algoritmos de pathfinding.

4.3.1 OpenGL API

O OpenGL é uma API de gráficos 3D amplamente utilizada para renderização de gráficos em tempo real (Vries, 2014). No desenvolvimento da infraestrutura gráfica para este estudo, o OpenGL foi escolhido por sua flexibilidade, desempenho e ampla adoção na indústria de jogos e simulações. A utilização do OpenGL também facilita a implementação de técnicas avançadas de renderização e otimização, garantindo que os testes sejam realizados em um ambiente gráfico realista.

4.3.1.1 Malhas

As malhas são uma representação comum de superfícies em computação gráfica, onde uma superfície é representada por um conjunto de vértices conectados por arestas e faces. No contexto deste estudo, as malhas serão utilizadas para representar os cenários de teste em que os algoritmos de pathfinding serão executados.

Cada conjunto de vértices e arestas diferentes são armazenados em *buffers* denominados VBO (Vertex Buffer Object) e EBO (Element Buffer Object), respectivamente, que são gerenciados pela GPU para renderização eficiente. Cada um desses *buffers* é associado a um VAO (Vertex Array Object), que encapsula o estado necessário para renderizar a malha, incluindo as ligações dos *buffers* e as configurações de atributos de vértice.

O conhecimento detalhado do funcionamento de cada buffer se prova essencial, quando consideramos a forma que o OpenGL lida com a renderização de malhas e troca de dados entre CPU e GPU. Uma vez que o OpenGL tem grandes problemas para lidar com múltiplas *threads*, o gerenciamento eficiente dos *buffers* e a minimização de operações de troca de dados entre CPU e GPU são cruciais para garantir o desempenho dos algoritmos de pathfinding em ambientes tridimensionais.

4.3.1.2 Programação Orientada a Eventos e Funções de *Callback*

Embora o OpenGL seja estritamente uma API de renderização, sem conhecimento nativo sobre o sistema operacional, janelas ou periféricos de entrada, a infraestrutura gráfica desenvolvida (IFCG) utiliza a biblioteca GLFW para o gerenciamento da janela e a criação do contexto gráfico. Essa integração permite implementar um modelo de programação orientada a eventos, onde o fluxo de execução é guiado por interações externas, como atualizações do sistema e entradas do usuário.

As funções de *callback* fornecidas pela API do GLFW foram implementadas na camada de gerenciamento do motor para interceptar eventos de teclado e mouse,

¹Disponível em: <https://github.com/andrevbastos/IFCG>. Acesso em: 22 mai. 2026.

servindo como uma ponte de comunicação com o sistema de renderização. Isso permite criar um ambiente de teste interativo e responsivo, onde as interações do usuário ditam de forma dinâmica as atualizações do cenário e os disparos de execução dos algoritmos de pathfinding em tempo real.

4.3.1.3 A Máquina de Estados e o Contexto OpenGL

Para lidar com a renderização em si, o OpenGL opera como uma vasta máquina de estados. Todas as configurações e referências de dados ficam armazenadas no que é chamado de Contexto OpenGL (fornecido e gerenciado pelo GLFW). Dessa forma, para renderizar uma malha, é necessário configurar o estado da máquina de acordo com as características do objeto — como a vinculação dos *buffers* VBO e EBO e a definição dos atributos de vértice — antes de emitir os comandos de desenho para a GPU.

Ademais, cada alteração na máquina de estados exige comunicação direta com o *driver* de vídeo. Como mudanças excessivas de estado geram alto custo de processamento (*overhead*), o gerenciamento eficiente dos *buffers* procura minimizar as trocas de estado e agrupar comandos sempre que possível, otimizando o tráfego de dados entre a CPU e a GPU.

4.3.1.4 Concorrência e Isolamento de *Threads*

Essa arquitetura baseada em contexto de estado impõe restrições rígidas à implementação de concorrência. O contexto do OpenGL é estritamente atrelado à *thread* em que foi ativado, tipicamente a *thread* principal. Tentativas de acessar ou modificar o estado da API gráfica a partir de múltiplas *threads* simultaneamente causam violações de acesso à memória, resultando em falhas críticas de limite de endereço, como erros SIGSEGV.

4.4 Geração de Cenários de Teste

Para garantir a diversidade e complexidade dos cenários de teste, é necessário implementar algoritmos de geração procedural de terrenos. Esses algoritmos permitem criar malhas 3D complexas e variadas, simulando diferentes tipos de ambientes que os algoritmos de pathfinding podem encontrar em situações reais.

A geração procedural é uma técnica utilizada para criar conteúdo de forma automática, utilizando algoritmos que produzem resultados variados e complexos a partir de um conjunto de regras ou parâmetros. No contexto deste estudo, a geração procedural será aplicada para criar terrenos que servirão como cenários de teste para os algoritmos de pathfinding.

4.4.1 Labirintos

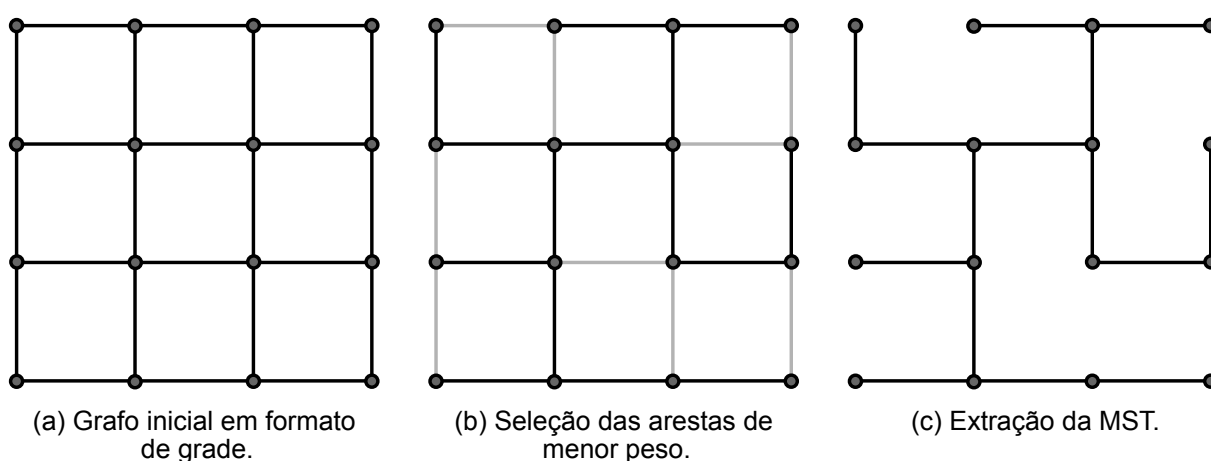
Temos como definição que labirintos são estruturas complexas compostas por caminhos interconectados, onde o objetivo é encontrar uma rota do ponto de partida

até um destino específico. A geração de labirintos pode ser realizada utilizando diversos algoritmos. Para este estudo, foi utilizado o algoritmo de Kruskal (Gomes, 2022), que é um método eficiente para gerar labirintos perfeitos, ou seja, labirintos sem ciclos, onde existe apenas um caminho entre quaisquer dois pontos.

Inicialmente, para gerar labirintos com o algoritmo de Kruskal, é construído um grafo em formato de grade (Figura Xa). Neste grafo, cada célula do labirinto é representada por um vértice, e as possíveis conexões entre as células são representadas por arestas de pesos aleatórios.

O algoritmo de Kruskal é então aplicado para construir uma *Minimum Spanning Tree* (MST), ou árvore geradora mínima, a partir desse grafo, selecionando as arestas de menor peso e garantindo que não sejam formados ciclos (Figura Xb). O resultado é um labirinto perfeito, onde cada célula tem um caminho que a conecta a qualquer outra célula, criando um caminho único entre o ponto de partida e o destino (Figura Xc).

Figura 3 – Etapas de geração de um labirinto perfeito utilizando o algoritmo de Kruskal



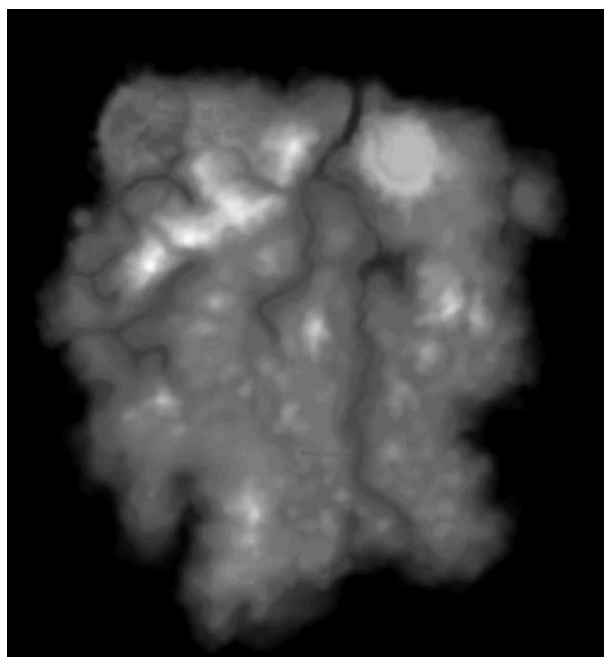
Fonte: Elaborado pelo autor (2026).

Com um labirinto perfeito em mãos, é possível sobrepor múltiplos labirintos perfeitos para criar cenários mais complexos. Essa técnica de sobreposição permite aumentar a densidade de obstáculos e criar topologias mais desafiadoras para os algoritmos de pathfinding, simulando ambientes mais realistas e variados.

4.4.2 Mapas de Altura

Uma mapa de altura nada mais é do que uma representação gráfica de um terreno, onde cada pixel da imagem representa uma coordenada no espaço tridimensional e a intensidade dos valores de cor indica a elevação do terreno naquela coordenada. Diversos desenvolvedores de jogos e simulações utilizam mapas de altura para criar terrenos realistas, como montanhas, vales e planícies. Oferecendo, na internet, uma variedade de cenários para testar os algoritmos de pathfinding.

Figura 4 – Exemplo de mapa de altura



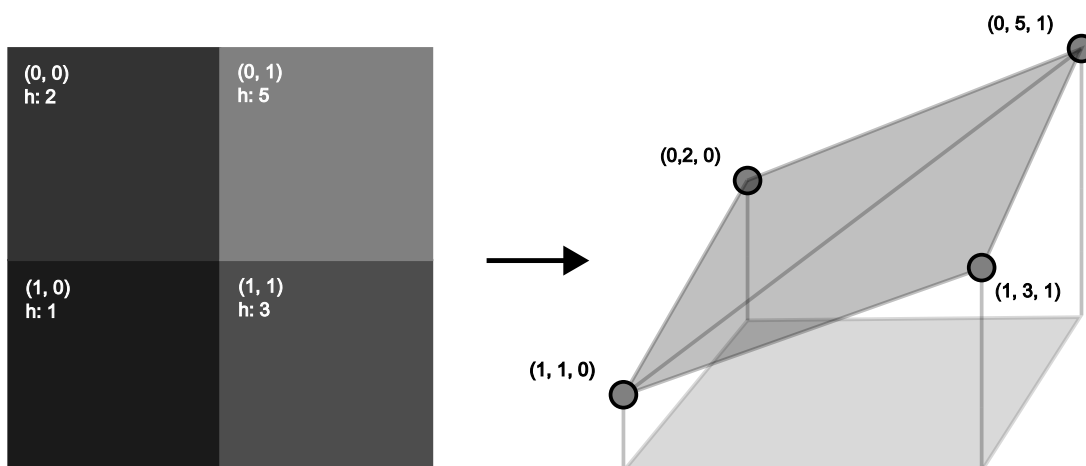
Fonte: The Imperial Library (2026).

Com isso, é possível gerar malhas 3D complexas a partir de simples imagens. Essa técnica é amplamente utilizada em jogos e simulações para criar ambientes naturais, como montanhas, vales e planícies, oferecendo uma variedade de cenários para testar os algoritmos de pathfinding.

É possível encontrar diversos mapas de altura gratuitos na internet, que podem ser utilizados para gerar malhas 3D realistas e complexas. Esses mapas de altura podem ser processados para extrair a geometria do terreno, criando uma malha que pode ser utilizada como cenário de teste para os algoritmos de pathfinding.

Para poder criar uma malha a partir dessas imagens, primeiro é necessário processar a imagem do mapa de altura para extrair as coordenadas dos vértices e suas respectivas alturas. Considerando i e j como os índices dos pixels da imagem, h como o valor de intensidade da cor do pixel e (x, y, z) como as coordenadas 3D de cada vértice da malha, é possível atribuir as coordenadas de cada vértice da malha com $(x, y, z) = (i, h, j)$.

Figura 5 – Conversão de um mapa de altura para uma malha 3D



Fonte: Elaborado pelo autor.

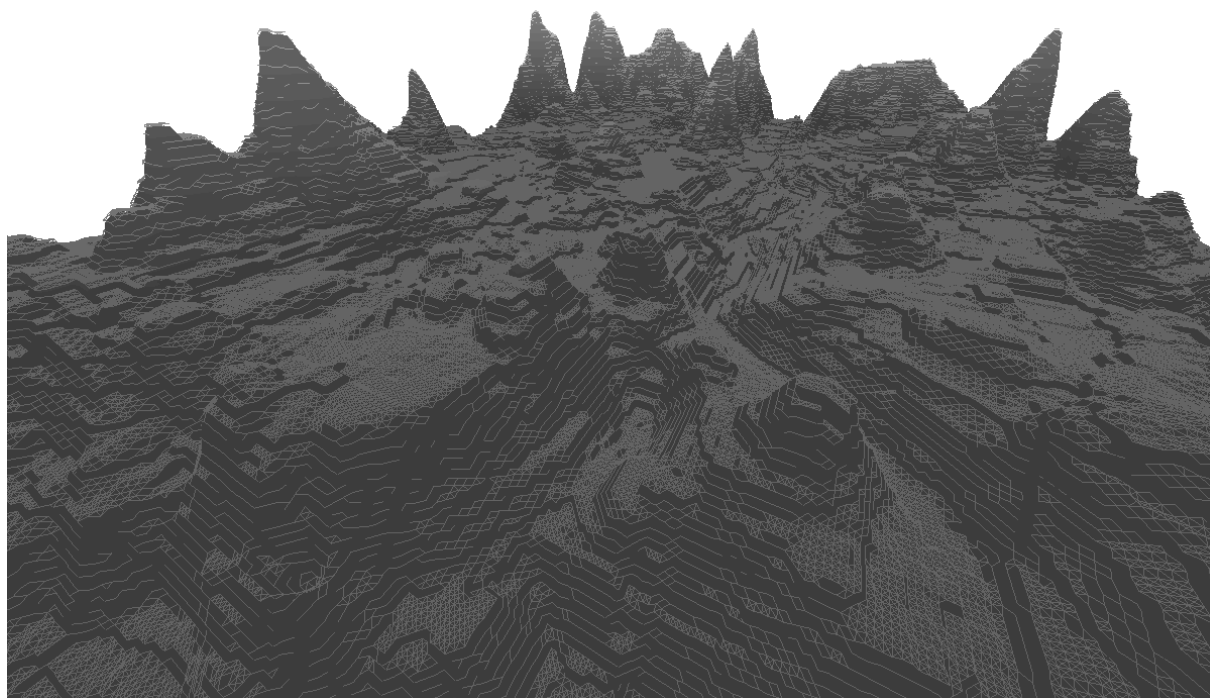
É importante ressaltar que os valores de cor (r, g, b) de uma imagem em escala de cinza variam igualmente de 0 a 255. Para evitar que as variações de altura no cenário 3D sejam desproporcionais às distâncias horizontais, o valor do pixel precisa ser normalizado e escalonado. Considerando v como o valor r, g ou b do pixel e H como a altura máxima desejada para o terreno, a elevação h de cada vértice é calculada pela fórmula:

$$h = \left(\frac{v}{255} \right) * H$$

Dessa forma, um pixel totalmente preto ($v = 0$) resultará em uma altura de 0, enquanto um pixel totalmente branco ($v = 255$) atingirá a altura máxima estipulada (H), permitindo um controle preciso sobre a amplitude do relevo gerado.

Por fim, são criadas faces conectando cada vértice com seus vizinhos diretos, formando uma malha de triângulos que representa a superfície do terreno. Essa malha pode então ser utilizada como cenário de teste para os algoritmos de pathfinding, permitindo avaliar seu desempenho em ambientes tridimensionais complexos.

Figura 6 – Exemplo de malha gerada a partir de um mapa de altura



Fonte: Elaborado pelo autor.

4.4.3 Geração de Ruídos

Utilizar mapas de altura foi uma grande vantagem para a geração de cenários realistas, mas é ineficiente procurar imagens pré-existentes para cada cenário de teste que se deseja criar. Para isso, é indispensável a implementação de algoritmos de geração de ruídos.

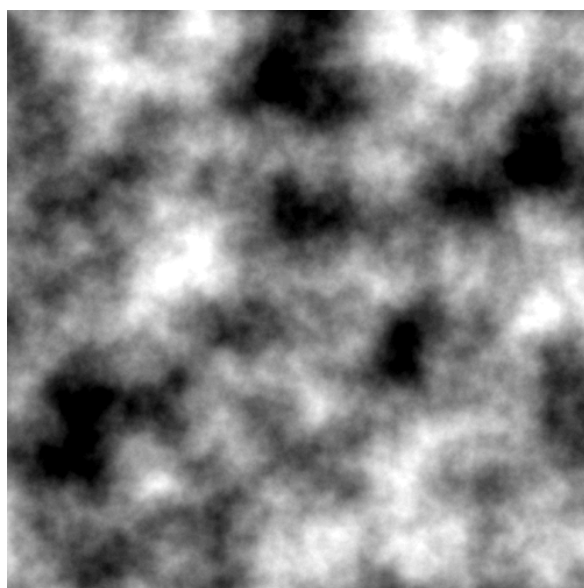
Um ruído é uma função matemática que gera valores pseudoaleatórios, mas de forma controlada, para criar padrões que se assemelham a fenômenos naturais. Existem diversos tipos de ruídos, como o ruído Perlin, o ruído Simplex e o ruído de valor, cada um com suas características e aplicações específicas. Porém, para essa pesquisa, o foco será no ruído Perlin, devido à sua capacidade de gerar padrões suaves e naturais, ideal para simular terrenos realistas.

O ruído de Perlin é um algoritmo de geração de ruído gradiente procedural que é amplamente utilizado para criar texturas e terrenos realistas em gráficos 3D. Ele foi desenvolvido por Perlin (1985) e é conhecido por produzir padrões de ruído suaves e naturais (Figura Xa), o que o torna ideal para simular superfícies como montanhas, nuvens e oceanos. Com isso em mente, podemos utilizar desse algoritmo para gerar nossos próprios mapas de altura, sem precisar recorrer a imagens pré-existentes. Isso nos dá controle total sobre a geração dos cenários de teste, permitindo criar uma

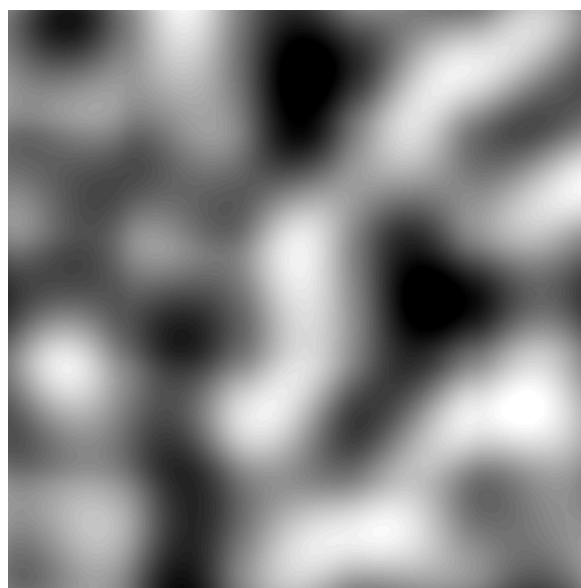
variedade de terrenos com diferentes características e desafios para os algoritmos de pathfinding.

Para conseguir um ruído suave e natural, o algoritmo gera diversas camadas de texturas diferentes (Figura Xb) e as sobrepõe, essas camadas são chamadas de *octaves* (oitavas). Ao sobrepor múltiplas *octaves*, é usada uma função de interpolação, que será detalhada na Seção 4.4.3.1, para suavizar a transição entre os valores gerados por cada camada, criando um resultado final que se assemelha a padrões naturais. A combinação de múltiplas *octaves* permite criar terrenos com detalhes variados, desde grandes elevações até pequenas variações de altura.

Figura 7 – Exemplo de ruído de Perlin e um de seus *octaves*



(a) Ruído de Perlin completo.

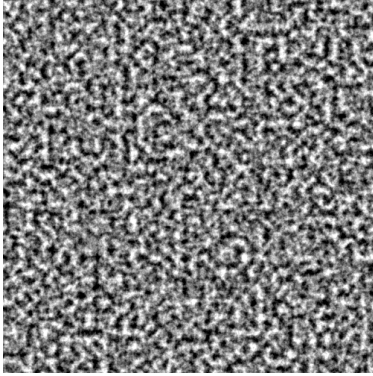


(b) Uma *octave* do ruído de Perlin.

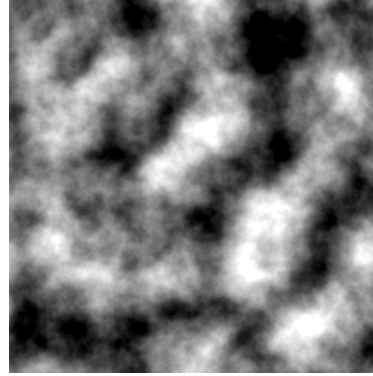
Fonte: Elaborado pelo autor.

Para gerar uma *octave* do ruído de Perlin, primeiro é necessário criar uma grade de tamanho unitário com vetores de gradiente unitários em cada ponto de interseção da grade, onde cada vetor aponta em uma direção aleatória. O tamanho de cada célula da grade determina o nível de detalhe do ruído, onde células maiores produzem um ruído mais suave, enquanto células menores geram um ruído mais detalhado.

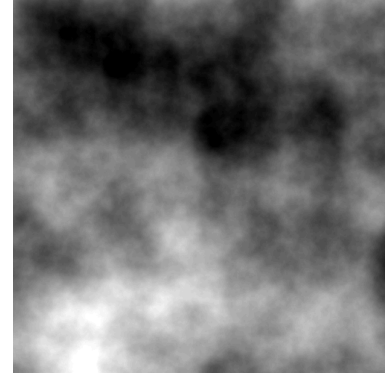
Figura 8 – Ruídos de Perlin com diferentes tamanhos de células



(a) Ruído gerado com células pequenas.



(b) Ruído gerado com células medianas.



(c) Ruído gerado com células grandes.

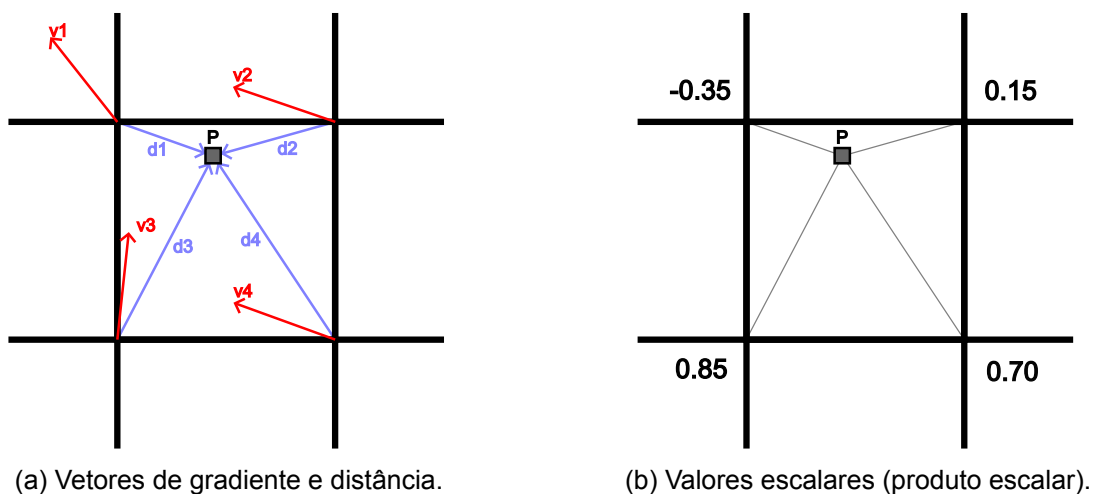
Fonte: Elaborado pelo autor.

Para cada ponto no espaço (ou *pixel*), o algoritmo calcula os vetores de distância entre ele e cada ponto de interseção de sua respectiva célula da grade. Em seguida, é calculado o produto escalar entre os vetores de distância $\vec{a} = (a_x, a_y)$ e os vetores de gradiente correspondentes $\vec{b} = (b_x, b_y)$, pela fórmula:

$$\vec{a} \cdot \vec{b} = (a_x * b_x) + (a_y * b_y)$$

O resultado do produto escalar é um valor numérico que representa a contribuição do vetor de gradiente para o *pixel* em questão. Por fim, é aplicada uma função de interpolação (Seção 4.4.3.1), onde cada valor escalar é suavizado para criar uma transição entre o *pixel* de forma ponderada, ou seja quanto mais próximo o *pixel* estiver do ponto de interseção, maior será a contribuição do vetor de gradiente para o valor final do ruído.

Figura 9 – Cálculo do produto escalar em uma célula do ruído de Perlin

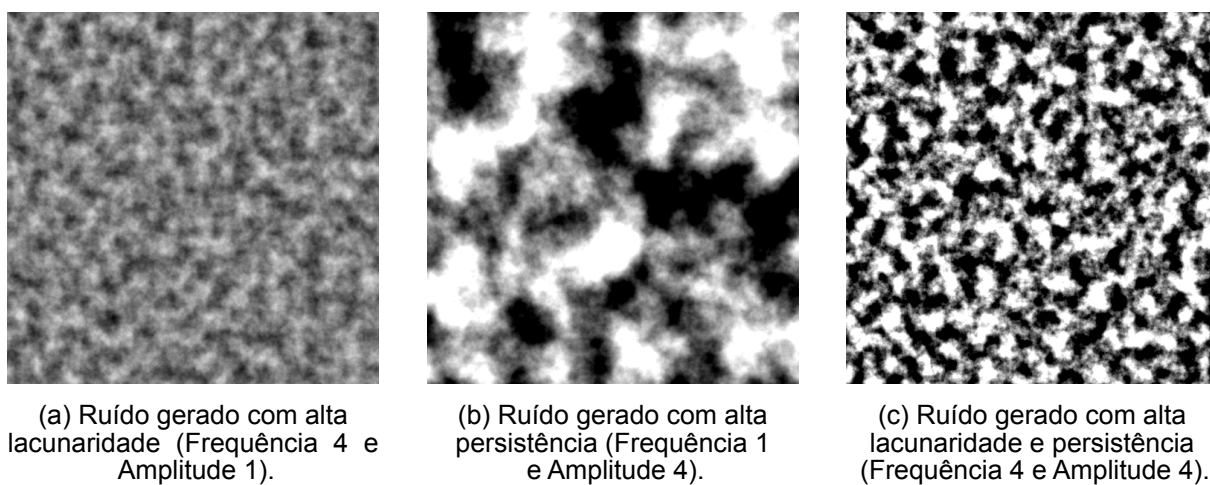


Fonte: Elaborado pelo autor.

Com uma *octave* do ruído de Perlin gerada, é possível sobrepor múltiplas octaves para criar um ruído mais complexo e detalhado. Porém, para evitar que as octaves sejam idênticas, é necessário aplicar valores que alterem sua geração. Para isso são introduzidos valores de frequência (Lacunaridade) e amplitude (Persistência), onde a frequência determina o número de detalhes presentes na *octave*, e a amplitude controla a intensidade desses detalhes.

Conforme Patel (2015) explica, a combinação de múltiplas octaves com diferentes frequências e amplitudes permite criar terrenos com uma variedade de características, desde grandes elevações até pequenas variações de altura, simulando ambientes naturais de forma realista.

Figura 10 – Ruídos de Perlin com diferentes frequências e amplitudes



Fonte: Elaborado pelo autor.

Assim, em cada ponto do espaço (x, y) , o valor base do ruído acumulado $V(x, y)$ é definido pela soma ponderada das *octaves*:

$$V(x, y) = \sum_{i=0}^{N-1} \left(a_i \cdot \text{perlin} \left(\frac{x \cdot f_i}{\lambda}, \frac{y \cdot f_i}{\lambda} \right) \right)$$

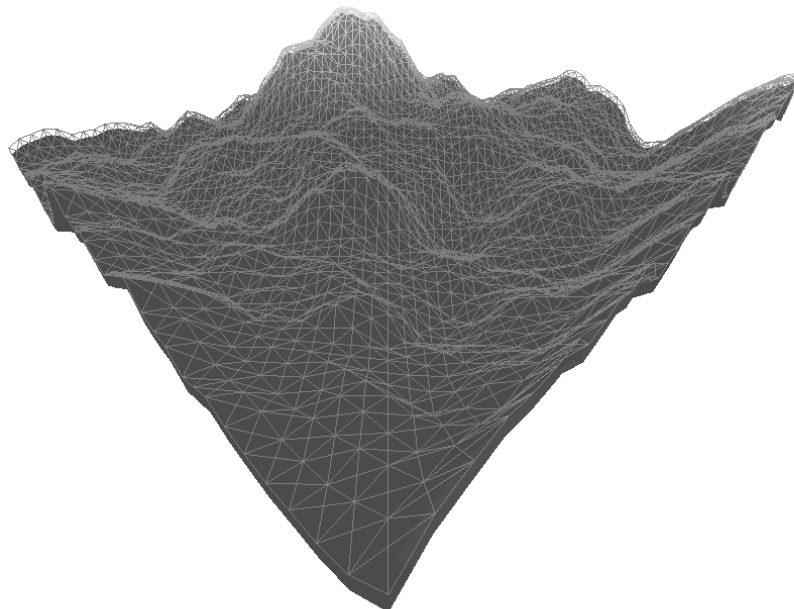
Onde N é o número de *octaves*, λ é o tamanho da célula, $f_i = F \cdot 2^i$ representa o aumento exponencial da frequência e $a_i = A \cdot 0.5^i$ representa o decaimento exponencial da amplitude.

Após o acúmulo, o valor final $V_f(x, y)$ do terreno é limitado para um intervalo de $[-1, 1]$ e pode ser remapeado para um intervalo de $[0, H]$, onde H é a altura máxima desejada para o terreno, utilizando a fórmula:

$$V_f(x, y) = \left(\frac{V(x, y) + 1}{2} \right) \cdot H$$

Finalmente, o valor $V_f(x, y)$ é utilizado para definir a elevação de cada vértice da malha como h . Isso permite gerar terrenos como feito anteriormente a partir de mapas de altura, mas com controle total sobre a geração dos cenários de teste, criando uma variedade de terrenos sem depender de imagens de terceiros.

Figura 11 – Exemplo de malha gerada a partir de um ruído de Perlin



Fonte: Elaborado pelo autor.

4.4.3.1 Função de Interpolação

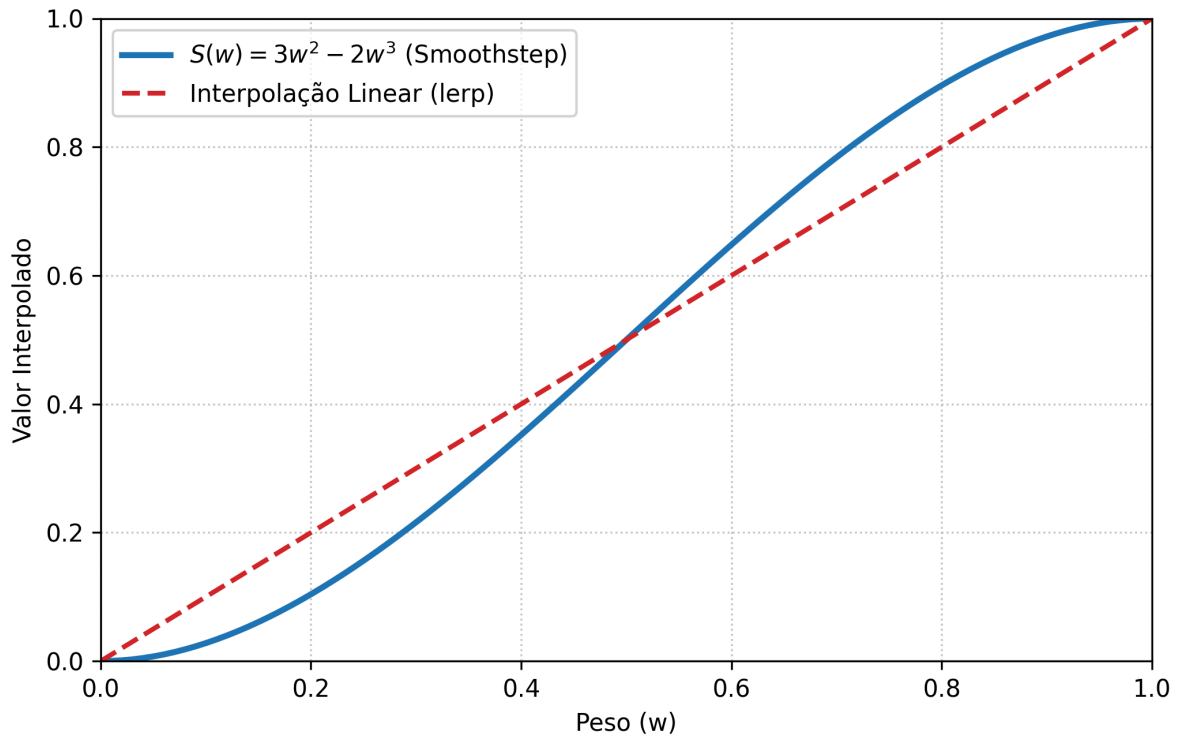
A função de interpolação é um componente crucial no algoritmo de geração de ruído de Perlin, pois é responsável por suavizar a transição entre os valores gerados por cada camada de ruído, criando um resultado final que se assemelha a padrões naturais.

A função de interpolação utilizada no ruído de Perlin é conhecida como função de suavização. Zipped (2023) ressalta que a interpolação linear simples não é adequada de forma isolada para gerar padrões naturais, pois deixa transições abruptas entre os valores de ruído, resultando em um terreno com bordas visíveis entre as células da grade.

Para solucionar esse problema e garantir uma transição orgânica, emprega-se uma curva de atenuação (ou função *fade*). No algoritmo original de Perlin, essa suavização é alcançada através de uma função polinomial cúbica, conhecida como *Smoothstep*, definida matematicamente por:

$$S(w) = 3w^2 - 2w^3$$

Figura 12 – Função de Suavização no Ruído de Perlin



Fonte: Elaborado pelo autor.

Essa curva mapeia o peso w (a posição relativa da coordenada, variando de 0 a 1) para um valor suavizado. A principal vantagem matemática do uso dessa função é que a sua primeira derivada é igual a zero em ambas as extremidades ($w = 0$ e $w = 1$). Isso garante que a transição de influência entre os gradientes de duas células vizinhas seja perfeitamente contínua, ocultando os limites da grade original.

Após obter o peso suavizado $S(w)$, o algoritmo realiza a interpolação linear padrão entre os valores de contribuição escalar a_0 e a_1 calculados:

$$f(a_0, a_1, w) = a_0 + (a_1 - a_0) \cdot S(w)$$

4.5 Algoritmos de Busca Heurística

4.5.1 Dijkstra

4.5.2 Heurísticas

4.5.3 A*

4.5.4 Jump Point Search (JPS)

4.5.5 Theta*

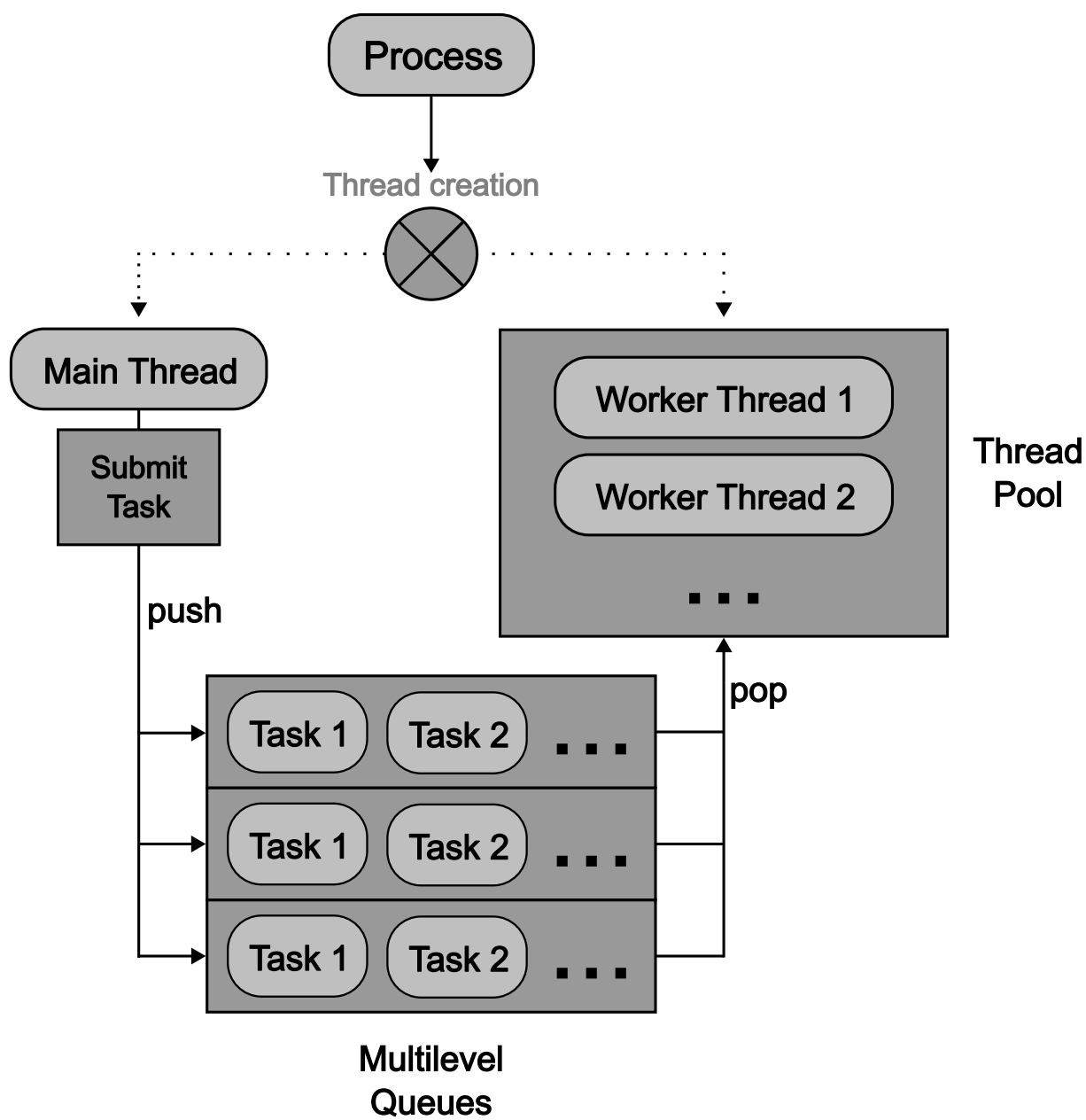
4.6 Concorrência e Paralelismo

4.6.1 *Multithreading*

4.6.2 Monitores

4.6.3 Fila Multinível e *Thread Pool*

Figura 13 – Diagrama de Thread Pool e filas multinível



Fonte: Elaborado pelo autor.

5 METODOLOGIA

Esta seção detalha a metodologia adotada para a realização da pesquisa, explicando como os algoritmos de pathfinding e a infraestrutura gráfica serão implementados e testados. Este projeto foi conduzido como uma pesquisa experimental de caráter quantitativo, sendo que a implementação da infraestrutura gráfica e dos algoritmos de pathfinding ocorreu em paralelo à coleta e análise de dados. A abordagem experimental permitirá a avaliação do desempenho dos algoritmos em condições controladas, enquanto a construção do motor do zero garantirá um ambiente de teste personalizado e otimizado para as necessidades específicas deste estudo.

5.1 Arquitetura do Motor Gráfico (IFCG)

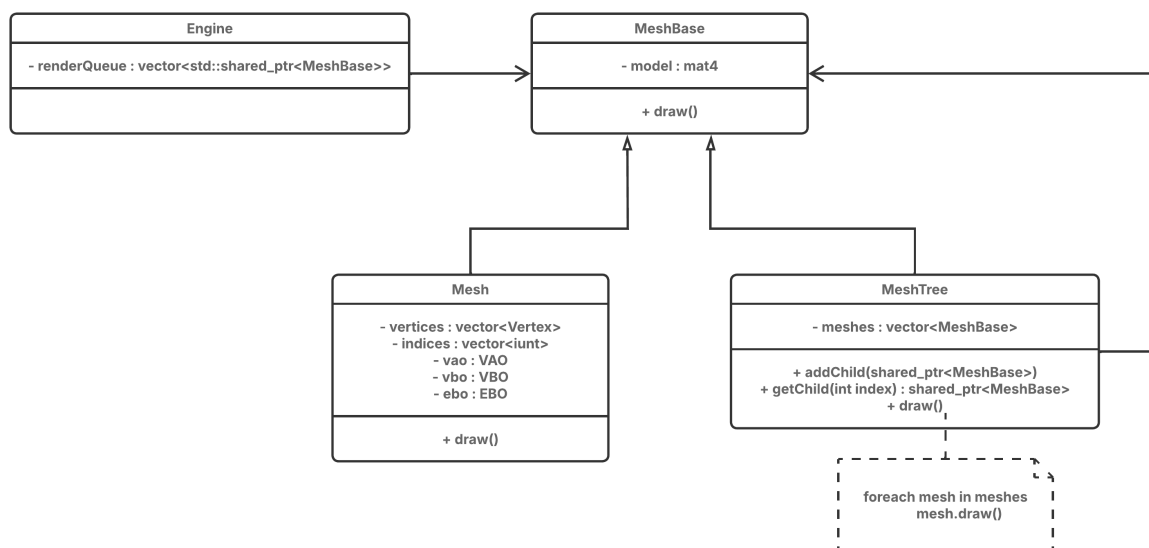
5.1.1 Aplicação de conceitos

5.1.2 Estrutura de Renderização e Gerenciamento de Recursos

A implementação do IFCG foi estruturada utilizando os princípios da programação orientada a objetos, para facilitar a organização do código, a reutilização de componentes e a manutenção do sistema. Além disso, foram aplicados diversos padrões de projeto para garantir a modularidade, flexibilidade e escalabilidade do código.

O padrão *Composite* (Gamma et al., 1994) foi utilizado para representar a hierarquia de objetos na infraestrutura gráfica, onde uma malha complexa pode ser composta por várias sub-malhas, e cada sub-malha pode ser tratada da mesma forma. Toda base de malha (*MeshBase*) contém sua própria matriz de modelo, que é responsável por armazenar as transformações de posição, rotação e escala da malha. Então as classes de malha (*Mesh*) e malha composta (*MeshTree*) herdam dessa base, permitindo que sejam tratadas de forma uniforme, independentemente de serem malhas simples ou compostas.

Figura 14 – Diagrama de classe do Composite



Fonte: Elaborado pelo autor.

Com essa arquitetura, o motor de renderização pode desenhar tanto malhas simples quanto compostas utilizando a mesma interface. Assim, permitindo criar cenários complexos com complexidade reduzida e mantendo a flexibilidade para adicionar novos tipos de malhas ou componentes gráficos no futuro.

O padrão *Singleton* foi aplicado na criação de instâncias do motor gráfico principal, garantindo um controle centralizado sobre os recursos gráficos e a renderização. Isso evita conflitos e mantém a consistência do ambiente de renderização, permitindo que diferentes partes do sistema acessem o motor gráfico de forma segura e coordenada.

5.1.3 Motor de Renderização e seus Componentes

5.2 Implementação da Estrutura de Grafos

5.2.1 Representação de Grafos Direcionados e Não Direcionados

5.2.2 Implementação de Algoritmos de Busca em Grafos

5.2.3 Otimizações e Estruturas de Dados Auxiliares

5.3 Geração e Processamento dos Cenários

5.3.1 Geração de Malhas 3D Complexas

5.3.2 Processamento de Malhas e Extração de Grafos

5.3.3 Configuração de Cenários de Teste e Parâmetros

5.4 Implementação dos Algoritmos e Sistema Concorrente

5.4.1 Implementação de Algoritmos de Pathfinding

5.4.2 Adaptação para Ambientes 3D

5.4.3 Implementação de Multithreading e Testes de Estresse

Para viabilizar a arquitetura da IFCG sem comprometer o desempenho, o ciclo principal de renderização e todas as chamadas ao OpenGL foram isolados na *thread* principal. Simultaneamente, a execução pesada dos algoritmos de pathfinding ocorre em *threads* operárias concorrentes (utilizando *jthread*). Os resultados desses cálculos são repassados à *thread* gráfica através de mecanismos seguros de sincronização de memória, permitindo que a geometria seja atualizada e renderizada sem corromper o contexto global da GPU.

5.5 Desenho Experimental e Coleta de Dados

5.5.1 Configuração dos Testes e Métricas Coletadas

5.5.2 Execução dos Testes e Registro dos Resultados

5.5.3 Análise Estatística e Visual dos Resultados

6 CRONOGRAMA

Tabela 1 – Cronograma de execução das atividades

ETAPAS	PERÍODOS
Revisão de Bibliografia	Jan/2026 a Fev/2026
Arquitetura Base (IFCG) e Estrutura de Grafos	Fev/2026 a Abr/2026
Heurísticas e Implementação Base (A*, Dijkstra, JPS, Theta*)	Mar/2026 a Mai/2026
Geração de Cenários de Teste e Malhas 3D	Abr/2026 a Mai/2026
Multithreading e Testes de Estresse	Mai/2026 a Jun/2026
Coleta de Dados Iniciais e Análise Estatística Simples	Mai/2026 a Jun/2026
Redação e Revisão do Pré-Projeto	Abr/2026 a Jun/2026
Defesa do Pré-Projeto	Jun/2026
Implementação de NavMeshes e Otimização 3D	Jul/2026 a Ago/2026
Otimização de Concorrência e Testes de Estresse	Ago/2026
Implementações Específicas (D* Lite, Busca Bidirecional, Lazy Theta*)	Ago/2026 a Set/2026
Coleta Automatizada de Métricas e Dados	Set/2026 a Out/2026
Análise Estatística e Visual dos Resultados	Out/2026 a Nov/2026
Redação da Monografia Final	Out/2026 a Nov/2026
Revisão e Entrega Final	Nov/2026
Defesa do TCC	Dez/2026

Fonte: Elaborado pelo autor (2026).

REFERÊNCIAS

GAMMA, Erich *et al.* **Design Patterns: Elements of Reusable Object-Oriented Software**. [S.l.]: Addison-Wesley, 1994.

GOMES, Paulo César Rodacki. **Grafos: conceitos fundamentais, algoritmos e aplicações**. Blumenau, SC: Editora do Instituto Federal Catarinense, 2022.

THE IMPERIAL LIBRARY. **Maps of Solstheim**. Disponível em: <https://www.imperial-library.info/content/maps-solstheim>.

PATEL, Amit J. **Making maps with noise functions**. Disponível em: <https://www.redblobgames.com/maps/terrain-from-noise/#elevation-redistribution>. Acesso em: 7 maio. 2026.

PERLIN, Ken. An Image Synthesizer. *In*: : SIGGRAPH '85. New York, NY, USA: Association for Computing Machinery, 1985. Disponível em: <https://doi.org/10.1145/325334.325247>

VRIES, Joey de. **Learn OpenGL: Enjoy learning modern OpenGL**. Disponível em: <https://learnopengl.com/>. Acesso em: 18 abr. 2026.

ZIPPED. **C++: Perlin Noise Tutorial**. Disponível em: <https://www.youtube.com/watch?v=kClAHqb60Cw>. Acesso em: 7 maio. 2026.